Big HW 2 ACG

OpenGL – 3D Mini-game

• For this project you will work in teams of 2-3 people.

• The homework **must be uploaded on Moodle** respecting the deadline. The homework must be submitted until the 17th of January 2023, 8:00. Late submissions will NOT be accepted. Projects that are not uploaded on Moodle will receive 0 points.

• Two identical projects found in different teams or any approximate project taken from the Internet **will be graded with 0.**

• The evaluation of the project will take place during the first lab after the submission deadline (lab 14). Presence at the lab will be **mandatory** for presenting your projects. If you are not present, you will get **0 points for it**.

• The final submission will be an archive with your VS project and it should contain a **README file** in which you will shortly specify all the functional sections of the hw. Additionally, if you have parts that don't work, you may offer solution ideas for a partial score.

• If you have any questions, please use the **special channel** created on teams (course ACG team, to have students from both computer science and electronics there). Don't forget to mention your lab assistant (e.g. @Iulia-CristinaStanica) so that we are notified about your question.

• You **MUST** use the **OpenGL game engine** available on Moodle. You **MUST** use **modern OpenGL** (this includes shaders, vbo, ibo, vao and so on). Deprecated OpenGL functions **are not allowed** in your code and will lead to the loss of all (or most of) the homework's points!

Tasks

By using **the game engine framework from lab 9 (and uploaded in the description of Big Hw 2)**, implement advanced functionalities required for a 3D mini-game. You **MUST** keep all modern OpenGL concepts (this includes shaders, vbo, ibo, vao and so on). Deprecated OpenGL functions are not allowed in your code.

Based on your scheduled lab, you will have to create a different world. For groups of mixed students (e.g. 2 students in Tuesday's group, 1 student in Wednesday's group), you will have to create the world corresponding to the lab of the majority of students (in this case, Tuesday).

World distribution:

- Tuesday 11:00 - Pompeii / Volcano
- Wednesday 12:00 - Underwater / Atlantis
- Thursday 10:00 - Magic forest
- Thursday 12:00 - Winter Wonderland
- Thursday 14:00 - Outer space environment
- Thursday 16:00 - Dune / Desert city

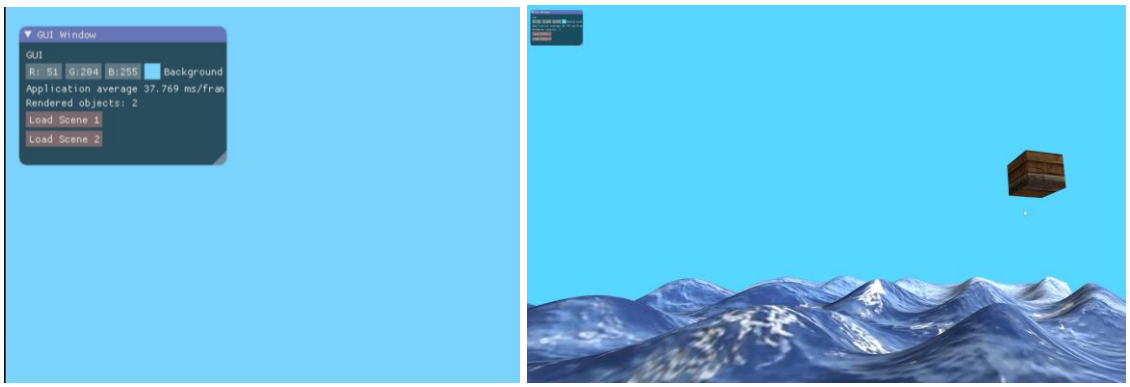
Create a 3D scene of your own liking. It should include at least:
- a **terrain** (mountains / hills), with an appropriate texture
- **multiple static objects** of your choice with textures or colors that can make your game look nice: trees, animals, buildings, characters, vehicles, any other element of décor that you find suitable (pay attention not to use elements that are very high poly because of performance limitations); the static objects should contribute to **creating the atmosphere / realism / fantasy of the world**
- **a dynamic effect / animation** of your choice (e.g. fish swimming, spaceship flying, day-night cycle with effect on the scene etc.) – create the animation in your scene over time OR make it happen when the user presses some keys

—----------

Add a player with first person perspective in the scene (**FPS camera attached to it)**. We must be able to move realistically in our scene so, based on your world, your player **should / should not fly** (make it translate and rotate only on the corresponding axis). Make sure to also include the **rotation** of the camera.

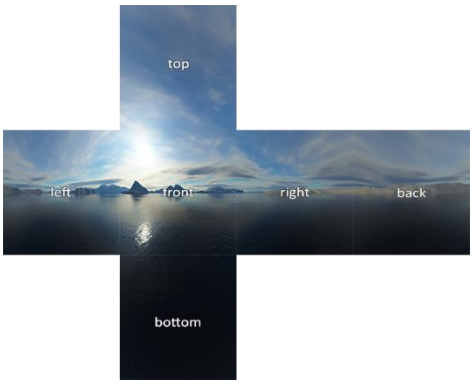
—----------

- Create a simple **2D GUI System**
- Display any information that you consider relevant (e.g. no of objects visible in the scene, window size etc.) in a 2D GUI window (example below)
- Include **at least one action element** (button, input text etc.) that performs a **change** in the scene (e.g. a button that will change the background if pressed)
- You can use any external library you want (e.g. Dear imgui - very easy to use)



—----------

Add a **skybox** in your scene corresponding to your scene (e.g. star sky, red sky etc.). You can use a cubemap for that (picture on the right).



—----------

- Create a **story** of your game and display it to the user 😀
  Based on the story, create at least **5 tasks** that your character should achieve in the game. The tasks should be distributed in **different areas** of your 3D scene.
- The tasks should be shown to the user one by one (either in the GUI or in the console). **Fulfilling a task should automatically show you the next one.**
- Each task should require some kind of **interaction with the environment** (on keyboard / mouse interaction OR by entering an area / hotspot) and **produce an effect** of your choice (e.g. picking up a magical apple from a tree, activating a lever that opens a secret door, stepping on a plate that deactivates a trap etc.). These effects should work ONLY if the user is in the proximity of the interaction spots.

Points:

1) (2.5p) Create a realistic scene, including terrain (0.5p) and various static objects as mentioned (1.5p), with corresponding textures. 0.5 points for making it look good
2) (1.5p) Create an animation of your choice
3) (0.5p) Make the correct changes and add the FPS camera of the player in your scene.
4) (1p) 2D GUI system (0.5 action element which performs changes)
5) (1p) Create the suitable skybox
6) (2.5p) Create at least 5 tasks with their corresponding interactions with the scene / objects (5x0.5p).

Points lost for:

- displaying all tasks from the start (-0.5p), not one by one after fulfillment
- tasks not distributed in different areas (and interaction not dependent on the player's position) -1p
- no story / connection between tasks -0.5p

1p for the app rendering with no errors and no crashing

IMPORTANT: In order to see DIFFERENT effects on various objects in the scene, you should create different shaders for them, based on their behavior (e.g. similar to our labs - animated objects, light source, mountains etc).

Bonus points:

- Handling player's **collisions** with objects in the scene (e.g. tree, stone etc.); on collision, moving in that direction is not possible anymore. You can use a very simple approach (e.g create bounding boxes or bounding spheres 0.5p) or more complex ones (octrees, BSP trees etc. 1p).
- Implement **water / fluids simulation** (lava etc.) using basic directional waves

(0.5p) or an advanced model (e.g. Gerstner, Perlin Noise, FFT 1p) and explain your method.

- Have multiple 3D scenes that we can load on button press (0.5p). Can increase based on the complexity of the scenes.
- BONUS (varying) for any other element that brings complexity and realism to the scene (e.g. 3D sounds)